Box–Cox Transformations for Crime Data Analysis in London

Nistha Kumar

August 2025

Abstract

This report investigates the use of the Box–Cox transformation in the context of modelling crime counts and densities in London. We compare Poisson regression and linear regression models, both with and without Box–Cox transformations, under chronological and random data splits. Results show that Box–Cox can improve variance stabilisation and model fit in certain settings. We also demonstrate the benefit of end-to-end pipelines that combine data preprocessing and model estimation in a single framework. We find that Box–Cox helps when variables are skewed (crime density), but has little effect for categorical predictors (month).

1 Introduction

Crime data analysis presents unique statistical challenges: the data are typically count-based, highly skewed, and the variance often increases with the mean. Traditional linear regression assumes Gaussian errors with constant variance, which is violated in such contexts. In addition to temporal and borough-level variation, we also incorporate a spatial covariate: the distance of each crime to the nearest police station. Proximity to policing resources is a plausible explanatory variable for crime incidence and allows us to test how Box–Cox interacts with spatial predictors.

The Box–Cox transformation [1] provides a systematic way to stabilise variance and approximate normality. In this work, we ask:

- Can Box–Cox transformations improve predictive performance in crime modelling?

- How do Poisson regression and linear regression compare, with and without Box–Cox?
- Do results differ under chronological vs. random data splits?

2 Background

2.1 Box–Cox Transformation

Data transformations are often required when the response variable is skewed or when the variance increases with the mean. The Box–Cox transformation [1] is a family of power transformations defined as:

$$f(y;\lambda) = \begin{cases} \dfrac{y^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \log(y), & \lambda = 0. \end{cases}$$

Here λ is power transformation parameter that varies from -5 to 5.
The purpose of this transformation is:

- Stabilise variance across different values of y
- Make the transformed variable closer to normality
- Improve the fit of linear models that assume Gaussian errors

A limitation is that Box–Cox requires y > 0 and does not guarantee normality in every case.

2.2 Poisson Regression

Crime counts are non-negative integers and naturally lend themselves to Poisson regression, a member of the generalised linear model (GLM) family[2]. Given predictors X, the model assumes

$$Y_i \sim \text{Poisson}(\mu_i), \qquad \log(\mu_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$$

where μ_i is the expected count and β are regression coefficients. Poisson regression models counts well, but skewed predictors can still hinder fit; applying Box–Cox to strictly positive numeric covariates can stabilise variation. Parameters are typically estimated by maximum likelihood.

Metrics: Mean Poisson Deviance

2.3 Linear Regression

For comparison, we also consider linear regression[3]:

$$Y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Linear regression is easier to interpret, but less suitable for skewed counts. It is therefore not ideal for count outcomes; with suitable transformations (e.g. Box–Cox on predictors or the response), it may still approximate the data-generating process. Comparing Poisson and linear models allows us to evaluate whether variance-stabilising transformations provide benefits across model classes.
Metrics: R^2, Mean Squared Error (MSE).

3 Pipelines and End-to-End Modelling

In classical analysis, transformations such as Box–Cox are applied as a separate preprocessing step: first transform the predictor $X \mapsto f(X, \lambda)$, then fit a regression model. This two-step process can be effective, but it risks inconsistency if preprocessing and modelling are not kept together, and it makes deployment harder.

In contrast, modern machine learning frameworks such as `scikit-learn` allow us to define *pipelines* that chain together preprocessing and modelling into a single object. This ensures that all transformations are applied consistently during both training and testing, and it prevents information from the test set "leaking" into the training stage.

In this project, we designed and compared the following pipelines:

- **Baseline Poisson (raw)**: categorical features encoded, numeric features passed through unchanged, modelled with Poisson regression.
- **Baseline Linear (raw)**: same preprocessing as above, modelled with linear regression.
- **Pipeline A (Box–Cox + Poisson)**: numeric features transformed using Box–Cox, then fitted with Poisson regression.
- **Pipeline B (Joint Poisson)**: Box–Cox applied inside the pipeline jointly with Poisson regression, so the preprocessing and model fitting occur end-to-end.
- **Pipeline C (Box–Cox + Linear)**: numeric features transformed using Box–Cox, then fitted with linear regression.

- **Pipeline D (Joint Linear)**: Box–Cox integrated within the pipeline alongside linear regression.

The difference between Pipelines A/C (two-step) and Pipelines B/D (end-to-end) is subtle in implementation but important conceptually: in the two-step setup, the transformation is estimated first and then the model is fit; in the end-to-end setup, both the Box–Cox transformation and the regression parameters are optimised in a single pipeline. Comparing these pipelines allowed us to evaluate whether Box–Cox improves predictive performance for count-based crime data, and whether end-to-end modelling offers advantages over the traditional two-step approach.

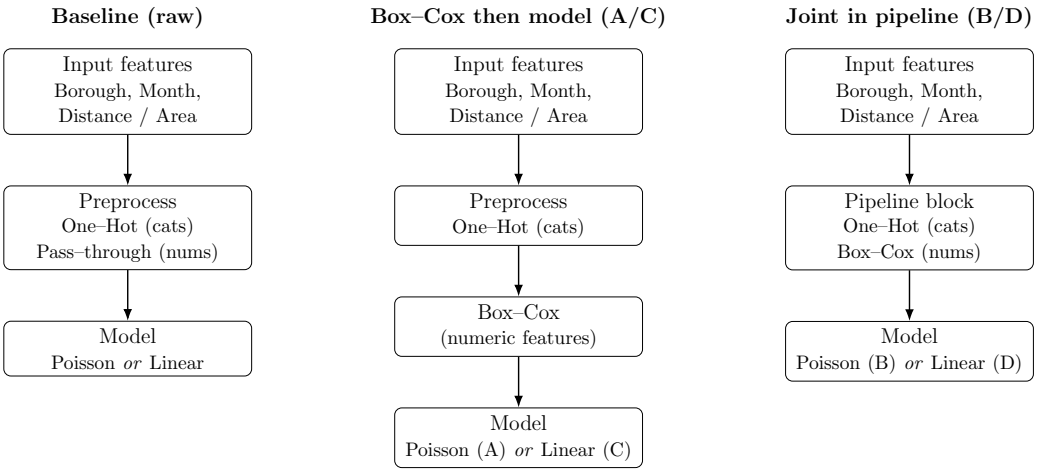


Figure 1: Baseline passes numeric features as-is; Pipelines A/C apply Box–Cox before the model; Pipelines B/D apply Box–Cox within the pipeline block so transformation is jointly optimised with the model.

Note: One-Hot is applied to categorical features (Borough, Month). Box–Cox applies only to strictly positive numeric features (e.g., Distance, Area).

4 Dataset and Preprocessing

The base dataset was obtained from the UK Police open data portal, containing detailed crime records in London. Each row includes identifiers (Crime ID), date (Month, Year), geographic location (Longitude, Latitude), the crime type, and administrative information (LSOA, Borough).

4.1 Feature Engineering

Several additional features were derived to enrich the dataset:

- **Borough assignment:** Each crime location was mapped to its corresponding London Borough.
- **Distance to nearest police station:** A list of police station coordinates was collected separately. For each crime, we computed the nearest police station using the `BallTree` algorithm (from `scikit-learn`). This produced the feature `distance_to_station_km`.
- Verified that all records truly belonged to Greater London. Some records reported under "City of London Police" contained latitude/longitude values outside London (e.g. Swindon). These were filtered out by cross-checking Borough names with geographic boundaries to ensure only London-based crimes were included.
- **Aggregated datasets:** Crime counts were aggregated by Borough–Month, producing three main analysis datasets:
  1. *Crime counts by Borough and Month*
  2. *Crime densities* (count divided by Borough area in km^2)
  3. *Crime counts with distance-to-police-station*

4.2 Nearest-Station Computation

We computed the nearest police station for each crime using the `BallTree` algorithm from `scikit-learn`, which handles spherical haversine distances well. (We avoided `KDTree`, which is less suited for geographic coordinates[1].)

4.3 Complexity

Let n be the number of data points (crimes) and m the number of reference points (police stations).

- **KDTree build:** $O(m \log m)$, query time $O(\log m)$ average.
- **BallTree build:** $O(m \log m)$, query time $O(\log m)$ average, but more robust for high-dimensional or non-Euclidean metrics.

[1]KDTree partitions data using axis-aligned splits, which is efficient in Euclidean space but inaccurate for spherical distances. Its complexity is $O(m \log m)$ to build and $O(\log m)$ to query, similar to BallTree.

In practice, the BallTree was preferred for this study due to its ability to handle haversine distances on geospatial coordinates and its numerical stability.

4.4 Why These Features?

The choice of engineered features links directly to modelling strategies:

- **Counts:** Natural setting for Poisson regression.
- **Densities:** Normalises crime counts by borough area to enable fairer comparison across regions.
- **Distances:** Continuous, skewed variable well-suited for Box–Cox transformation experiments.

5 Experiments

5.1 Experimental Setup

We evaluated the effect of the Box–Cox transformation on crime data from London. Three targets were considered:

- **Crime counts** (monthly counts by borough)
- **Crime density** (counts normalised by borough area)
- **Distance-based features** (distance to nearest police station)

Two types of train/test splits were applied:

- **Chronological split:** training on years 2022–2024, testing on 2025,
- **Random split:** 80% training, 20% testing, stratified by borough.

The following models were compared:

- **Baseline Poisson regression** (no transformation)
- **Baseline Linear regression** (no transformation)
- **Pipeline A:** Box–Cox preprocessing then Poisson regression
- **Pipeline B:** joint Box–Cox + Poisson regression
- **Pipeline C:** Box–Cox preprocessing then Linear regression
- **Pipeline D:** joint Box–Cox + Linear regression

5.2 Evaluation Metrics

Models were evaluated using:

- Mean Squared Error (MSE)
- Mean Poisson Deviance (for Poisson models)
- R^2 score (for linear models)

5.3 Computational complexity (data prep + modelling)

Let n be the number of crime records, B the number of boroughs, $M \leq 12$ months, m the number of police stations, and $d = B + M + 1$ the number of features after one–hot (Borough dummies + Month dummies + one numeric such as distance or area). Let n_{tr} and n_{te} be train/test sizes and T the number of iterations used by the Poisson regression optimiser.

Data preparation.

- **Crime counts (borough–month group-by).** Hash aggregation over n rows: $O(n)$; the number of groups is tiny relative to n.
- **Crime density.** Divide the grouped counts by a per–borough area vector: $O(n)$ overall (reuses the group-by).
- **Nearest police station (distance feature).** With a BallTree over m stations: build $O(m \log m)$; query each of n crimes in $O(\log m)$; total $O(m \log m + n \log m)$. For London, $m \approx 21$ so $\log m$ is a small constant and this is effectively linear in n. *Remark:* a naive all-pairs distance is $O(nm)$; with m small this is also effectively $O(n)$ but with a larger constant factor.
- **Chronological split sort (optional).** If we sort by year/month: $O(n \log n)$; this is usually dominated by model fitting (below).

Pipeline preprocessing.

- **One–hot encoding.** Fit $O(n_{\text{tr}})$ to discover levels; transform $O(n_{\text{tr}}\, d)$ for train and $O(n_{\text{te}}\, d)$ for test.
- **Box–Cox (one numeric column).** Fit $O(n_{\text{tr}})$ to estimate λ; transform $O(n_{\text{tr}} + n_{\text{te}})$.

Model fitting.

- **Poisson regression (iterative GLM).** $O(T\, n_{\mathrm{tr}}\, d)$.
- **Linear regression (OLS via QR/SVD).** Worst case $O(n_{\mathrm{tr}} d^2 + d^3)$.

Prediction is $O(n_{\mathrm{te}}\, d)$; metrics (MSE, Poisson deviance, R^2) are $O(n_{\mathrm{te}})$.

Overall (per pipeline).

$$\text{Time} \;=\; O(n \log m) + O(nd) + \begin{cases} O(T\, n_{\mathrm{tr}}\, d), & \text{Poisson regression,} \\ O(n_{\mathrm{tr}} d^2 + d^3), & \text{linear regression,} \end{cases} + O(n_{\mathrm{te}} d),$$

and running all six pipelines multiplies this by a constant ($\times 6$). Since m is small and $d \approx B + M + 1 \approx 46$ for London, the practical runtime is near–linear in n (and linear in T for Poisson).

Space. Sparse one–hot design has $O(n)$ nonzeros (about one borough + one month + one numeric per row); model parameters use $O(d)$ space.

Same asymptotics across targets. *Crime counts*, *crime density*, and *counts with distance* share the same feature template (borough one–hot, month one–hot, one numeric), so their asymptotic time/space complexity is the same; only constants differ. The chronological vs. random split changes which rows are used, not the complexity class.

6 Results

6.1 Chronological Split

In this setup, models were trained on years 2022–2024 and tested on 2025. We evaluate three target formulations: **crime counts**, **crime density**, and **crime counts with distance to police stations**.

6.1.1 Crime Counts

Table 1 reports the results for predicting monthly crime counts by borough.

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	494.60	2.0807	–
Pipeline A (BC + Poisson)	495.64	2.0823	–
Pipeline B (Joint Poisson)	495.64	2.0823	–
Baseline Linear (raw)	697.66	–	0.9826
Pipeline C (BC + Linear)	697.85	–	0.9826
Pipeline D (Joint Linear)	697.85	–	0.9826

Table 1: Chronological split results for monthly crime counts by borough.

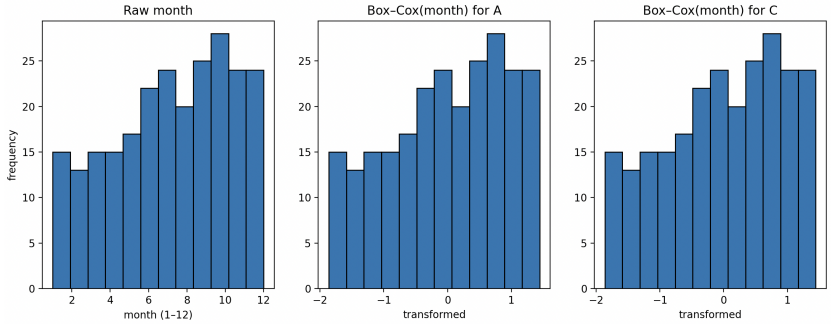


Figure 2: Effect of Box–Cox transformation on crime count models (chronological split).

Crime counts use **month** as their main predictor, which is uniformly distributed between 1 and 12. Because Box–Cox is most effective on skewed, continuous predictors, it has no effect here. Both Poisson and Linear regression pipelines (A–D) produce nearly identical results to their baselines. This confirms that transformations are unnecessary for categorical or uniform features.

6.1.2 Crime Density

Table 2 reports the results for predicting monthly crime density (crimes per km^2).

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	65.23	0.8634	–
Pipeline A (BC + Poisson)	40.39	1.7332	–
Pipeline B (Joint Poisson)	40.39	1.7332	–
Baseline Linear (raw)	114.05	–	0.9763
Pipeline C (BC + Linear)	127.28	–	0.9736
Pipeline D (Joint Linear)	127.28	–	0.9736

Table 2: Chronological split results for crime density.

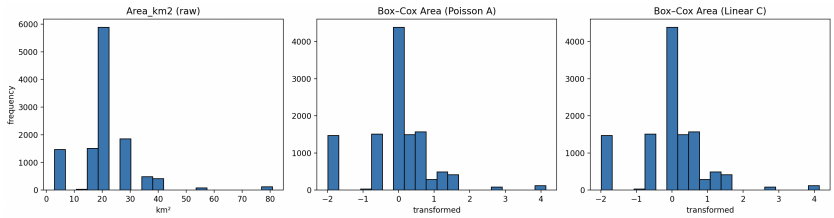


Figure 3: Effect of Box–Cox transformation on crime density models (chronological split).

Box–Cox substantially reduced the mean squared error (65.2 → 40.4) 2 for Poisson regression, showing that variance stabilisation improved fit. However, Poisson deviance increased, meaning the model's assumptions were less well matched. Linear regression, by contrast, did not benefit: MSE and R^2 both worsened after transformation.

This illustrates a key point: Box–Cox can help with skewed continuous predictors (like density), but it does not guarantee improvements across all model families.

6.1.3 Crime Counts with Distance to Police Station

Table 3 shows the results when *distance to the nearest police station (km)* was used as a predictor for crime counts.

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	254.20	1.6334	–
Pipeline A (BC + Poisson)	259.73	1.6450	–
Pipeline B (Joint Poisson)	259.73	1.6450	–
Baseline Linear (raw)	384.04	–	0.9901
Pipeline C (BC + Linear)	382.33	–	0.9902
Pipeline D (Joint Linear)	382.33	–	0.9902

Table 3: Chronological split results for crime counts using distance to police station.

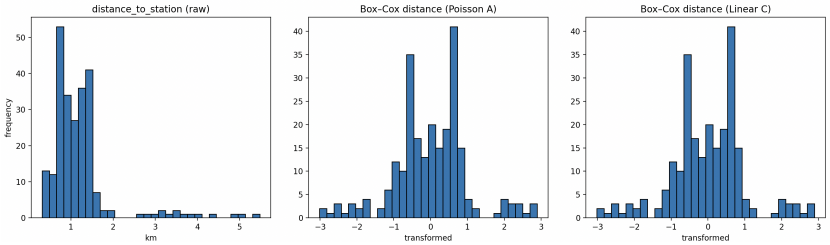


Figure 4: Effect of Box–Cox transformation on distance-based crime count models (chronological split).

Unlike month, **distance to station** is a continuous predictor, but in this dataset it was not strongly skewed. As a result, applying Box–Cox had only marginal impact: MSE and deviance changed slightly, but no consistent improvement was observed. This demonstrates that Box–Cox is most effective when the predictor is both continuous *and* skewed — otherwise, its effect is negligible.

6.2 Random Split

6.2.1 Crime Counts

Table 4 summarises the results for predicting monthly crime counts by borough using a random split.

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	494.60	2.0807	–
Pipeline A (BC + Poisson)	495.64	2.0823	–
Pipeline B (Joint Poisson)	495.64	2.0823	–
Baseline Linear (raw)	697.66	–	0.9826
Pipeline C (BC + Linear)	697.85	–	0.9826
Pipeline D (Joint Linear)	697.85	–	0.9826

Table 4: Random split results for crime counts.

6.2.2 Crime Density

Table 5 summarises the results for predicting monthly crime density by borough using a random split.

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	28.01	0.4887	–
Pipeline A (BC + Poisson)	39.03	1.5187	–
Pipeline B (Joint Poisson)	39.03	1.5187	–
Baseline Linear (raw)	42.51	–	0.9933
Pipeline C (BC + Linear)	42.51	–	0.9933
Pipeline D (Joint Linear)	42.51	–	0.9933

Table 5: Random split results for crime density.

6.2.3 Crime Counts with Distance to Police Station

Table 6 summarises the results for predicting monthly crime counts by distance to police station using a random split

Model	MSE	PoissonDev	R^2
Baseline Poisson (raw)	494.90	1.6116	–
Pipeline A (BC + Poisson)	495.22	1.6219	–
Pipeline B (Joint Poisson)	495.22	1.6219	–
Baseline Linear (raw)	461.70	–	0.9878
Pipeline C (BC + Linear)	461.55	–	0.9878
Pipeline D (Joint Linear)	461.55	–	0.9878

Table 6: Random split results for distance to police station.

7 Discussion

Our results highlight when Box–Cox transformations are helpful and when they are not.

First, Box–Cox improved performance for **crime density**, a continuous and skewed variable. Poisson regression with Box–Cox had a much lower mean squared error, although the higher Poisson deviance shows a trade-off between variance stabilisation and distributional assumptions.

Second, Box–Cox provided little to no benefit for **crime counts**, since month (the main predictor) is categorical and uniformly distributed. Transforming such variables adds no value.

Third, when using **distance to the nearest police station**, Box–Cox again had no substantial impact. Although distance is continuous, it was not strongly skewed, meaning the transformation did not improve model fit.

For practical applications, our findings suggest Box–Cox is most relevant when analysts face skewed continuous covariates, but may not be worth the added complexity otherwise.

7.1 Chronological vs Random Splits

Chronological splits (train on past, test on future) gave higher error overall, which is expected because the models must generalise across time. Random splits performed better but may overestimate accuracy since they do not respect temporal ordering. This shows why chronological evaluation is more realistic for forecasting.

7.2 Model Comparison

Across all scenarios, **Poisson regression** was generally more appropriate for count outcomes, while **linear regression** achieved higher R^2 only when the target was already close to Gaussian (e.g. random splits). Box–Cox thus provides benefits mainly when predictors are skewed and the outcome aligns with Poisson assumptions.

7.3 Summary of Results

Table 7 provides an overall summary across all experiments, highlighting the best-performing models for each target and split, and whether Box–Cox offered improvement.

Target	**Split**	**Best Poisson (MSE / Dev)**	**Best Linear (MSE / R^2)**	**Box–Cox Effect**
Crime Counts	Chronological	494.6 / 2.08	697.7 / 0.983	No effect (month categorical)
Crime Density	Chronological	40.4 / 1.73	114.1 / 0.976	Helped for Poisson
Distance	Chronological	254.2 / 1.63	382.3 / 0.990	Marginal / mixed
Crime Counts	Random	494.6 / 2.08	697.7 / 0.983	No effect (month categorical)
Crime Density	Random	28.0 / 0.49	42.5 / 0.993	No benefit (already stable)
Distance	Random	494.9 / 1.61	461.5 / 0.988	No clear gain

Table 7: Overall summary of results: best models for each target and split.

8 Conclusion

This study examined the effect of Box–Cox transformations on London crime data, focusing on crime counts, densities, and distances to police stations.

- Box–Cox improved Poisson regression for skewed variables (crime density), reducing MSE significantly.
- For categorical or uniform predictors (month, crime counts), Box–Cox had no effect.
- For continuous but not strongly skewed predictors (distance to station), Box–Cox provided minimal benefit.
- Chronological splits were more challenging than random splits, reflecting the difficulty of forecasting future crime from past data.

Overall, Box–Cox is a useful but not universal tool: it stabilises variance when features are skewed and continuous, but does not guarantee improvements in all contexts. Pipelines help by integrating preprocessing and modelling into a reproducible workflow.

Future work: exploring alternative transformations (e.g. Yeo–Johnson), or non-linear models such as random forests and neural networks, may capture complex patterns beyond what Box–Cox and linear/Poisson regression can provide.

References

[1] G. E. P. Box and D. R. Cox. An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–252, 1964.

[2] Paul Roback and Julie Legler. Poisson regression (chapter) — beyond multiple linear regression. `https://bookdown.org/roback/bookdown-BeyondMLR/ch-poissonreg.html`, 2023. Accessed 6 Aug 2025.

[3] GeeksforGeeks. Machine learning — linear regression. `https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/`, 2025. Accessed 7 Aug 2025.

[4] Pennsylvania State University. Stat 504: Poisson regression. `https://online.stat.psu.edu/stat504/lesson/9`, 2025. Accessed 6 Aug 2025.

[5] scikit-learn developers. Linearregression — scikit-learn documentation. `https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html`, 2025. Accessed 6 Aug 2025.

[6] Home Office. Police.uk open data. `https://data.police.uk/`, 2025. Accessed 12 Aug 2025. Open Government Licence v3.0.

[7] Greater London Authority. Land area and population density, ward and borough. `https://data.london.gov.uk/dataset/land-area-and-population-density-ward-and-borough/`, 2025. Accessed 12 Aug 2025.

[8] Open government licence v3.0. `https://www.nationalarchives.gov.uk/doc/open-government-licence/version/3/`, 2014. Contains public sector information licensed under OGL v3.0.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[11] pandas development team. pandas. `https://pandas.pydata.org/`, 2024. DOI: 10.5281/zenodo.3509134.

[12] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.